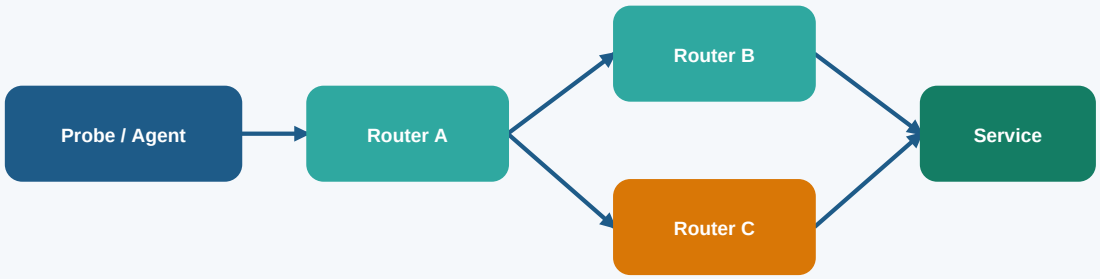
AIORI HACKATHON 3

Evidence-Based Operator Network Fault Localisation

PS-035 | Technical Solution Proposal



Illustrative topology: fault is inferred from multiple independent observations, not one failed probe.

Team	TEJAS
Mentor	Vinay
Contributors	Pratyaksh · Backend | Badal Saxena · Frontend & UI/UX
Institution	INVERTIS UNIVERSITY

A practical, evidence-led approach to turn network symptoms into ranked fault hypotheses that operators can inspect and verify.

Abstract & Problem Description

Abstract. Network incidents are often observed through separate signals: failed reachability checks, increased round-trip time, packet loss, DNS resolution failures and incomplete traceroutes. Looking at one signal in isolation can identify a symptom without identifying the likely fault domain. PS-035 proposes a monitoring and decision-support system that collects permitted active probes, aligns them with topology and historical baselines, correlates evidence, and ranks suspected links or devices with an explanation. The dashboard will show the affected target, candidate fault location, supporting observations, confidence band, timestamps and recommended verification steps. The prototype is designed to assist operators—not to claim certainty from incomplete measurements.

Problem being addressed

- **Fragmented evidence:** operators may need to inspect several tools and logs before forming a coherent incident view.
- **Ambiguous symptoms:** a server timeout can be caused by the endpoint, DNS, access policy, congestion, routing or an intermediate link.
- **Topology matters:** the same symptom observed by multiple probes can help distinguish a shared upstream fault from a target-specific failure.
- **False certainty risk:** a missing traceroute hop or unanswered ICMP echo is not, by itself, proof that a router is down.
- **Operational need:** present the likely fault domain together with evidence and limitations so a human operator can verify the hypothesis.

Project objectives

- Collect and timestamp reachability, latency, packet-loss, DNS, TCP/HTTP and route observations where permissions and tools allow.
- Represent known devices, links and target dependencies as a graph.
- Correlate temporal and topological evidence to rank candidate fault locations.
- Expose transparent explanations and evidence provenance in a web dashboard.
- Evaluate detection, localisation, false alarms and response time on controlled, reproducible test scenarios.

Scope boundary: The initial prototype localises faults only as precisely as the available probes and topology allow. Device-interface diagnosis may require SNMP/streaming telemetry, logs, or operator-provided data that is not available from ping/traceroute alone.

Background

Network fault localisation is the task of narrowing observed service degradation to a plausible component or fault domain. Monitoring typically combines active measurements (traffic generated by probes) with passive/device-side observations (interface counters, logs, flow records or telemetry). This proposal starts with active probing because it is comparatively easy to demonstrate in a controlled prototype; device-side signals are an optional enhancement when authorised access exists.

Evidence sources and what they can—and cannot—tell us

Signal	Useful evidence	Important limitation
ICMP echo / ping	Reachability, round-trip time, loss over repeated samples	ICMP may be filtered, rate-limited or deprioritised; failure is not proof of host failure.
Traceroute / TTL probes	Visible hop sequence and where responses stop changing	Routers may not return TTL-expired replies; paths may be asymmetric or load-balanced.
DNS query	Resolution success, response code, answer and timing	Failure may be resolver, zone, delegation, policy or name-specific—not a transit-link fault.
TCP / HTTP probe	Port/service reachability, response code, TLS/application-level result	A successful ICMP probe does not guarantee the application works; application checks need correct expectations.
Latency / loss history	Deviation from a baseline, duration and scope of degradation	Thresholds vary by service and path; short windows can be noisy.
Device telemetry (optional)	Interface errors, discards, utilisation, device health	Needs authorised access and consistent device identifiers/time alignment.

Proposed operating principle

Treat every measurement as an observation with a timestamp, probe source, target, method, result and quality flag. A fault hypothesis should be strengthened by independent, mutually consistent observations—for example, multiple downstream targets degrade while targets on an alternate branch remain healthy—and weakened by contradictory evidence. The system should preserve raw observations so that the explanation can be audited.

Terminology

- **Symptom:** what the probe observed, such as elevated latency or DNS timeout.
- **Candidate fault:** a link, device, resolver or endpoint that could explain the symptom.
- **Confidence:** a calibrated score or qualitative band based on evidence coverage and consistency—not a guarantee.
- **Fault domain:** the narrowest area justified by available evidence, which may be a path segment rather than a specific router.

Literature & RFC Review

This review focuses on standards and operational tooling directly relevant to measurement and evidence collection. These references define measurement mechanisms or probe capabilities; they do not prescribe the project's proposed ranking algorithm.

Relevant standards and tools

Reference	Relevance to PS-035	How we use it
RFC 792 — ICMP [1]	Defines Internet Control Message Protocol messages, including echo and error reporting.	Basis for ICMP reachability probes and interpreting network-layer responses.
RFC 1034 / RFC 1035 — DNS [2,3]	Core DNS concepts and protocol/implementation specification.	Grounds DNS checks, response-code recording and resolver/name-related fault hypotheses.
RFC 4656 — OWAMP [4]	Standardised one-way delay and loss measurement protocol.	Reference for advanced measurement; not required for the minimum demo.
RFC 5357 — TWAMP [5]	Standardised two-way / round-trip performance measurement based on OWAMP.	Potential future integration when controlled measurement endpoints are available.
Prometheus Blackbox Exporter [6]	Operational tooling supports HTTP, HTTPS, DNS, TCP, ICMP and gRPC probes.	Possible adapter for repeatable probe collection and timing metrics; integrate only if deployment permits.
Traceroute / TTL-based probing	Uses hop-limited probes and response messages to infer visible path hops.	Collect hop sequences as evidence, while treating non-response as ambiguous.

Research gap and proposed contribution

The standards above provide measurement primitives; the project contribution is the evidence-correlation layer and operator-facing explanation. The system will combine observations across targets, probe types and time windows with a topology graph, then return ranked candidate fault domains. This should be presented as a prototype design contribution, not as a claim that the RFCs themselves specify root-cause inference.

Design constraints from the review

- Measurements need timestamps and repeatable probe settings to compare results fairly.
- DNS, transport, network reachability and application availability should be diagnosed separately before being correlated.
- Measurement agents must be authorised and rate-limited; probes can be blocked or treated differently by networks.
- The system must show missing/ambiguous evidence rather than silently converting it into a failed-device label.

Proposed Solution

The proposed system is a web-based network fault localisation assistant. It collects network observations, normalises them into a common event format, evaluates their relationship to a known topology, and presents explainable candidate faults. The first version can use controlled lab nodes or a small allowlisted set of endpoints.

Core workflow

- **1. Configure scope:** operator adds authorised targets, probe locations, topology links, service expectations and probe intervals.
- **2. Collect observations:** execute ICMP/reachability, latency, packet-loss, DNS, TCP/HTTP and traceroute checks supported by the environment.
- **3. Normalize:** record UTC timestamp, source, target, method, result, duration, error/response code and collection status.
- **4. Build context:** map each target to known path segments, devices and dependencies; retain uncertainty where paths are unknown or dynamic.
- **5. Correlate evidence:** compare recent observations to baselines, examine affected-target overlap, and identify the earliest common suspect segment supported by the data.
- **6. Rank and explain:** output candidate, evidence for/against, evidence freshness, confidence band and suggested validation action.
- **7. Alert and review:** dashboard shows incident timeline, topology overlay, raw evidence and operator acknowledgement.

Illustrative incident

Observation: Service S becomes slow/unreachable. Probes to S and another target behind Router C show increased latency and loss; a target reachable through Router B remains normal. DNS resolves correctly; TCP/HTTP checks fail only for the affected branch. A traceroute response disappears near the Router A → Router C segment.

System output: “Candidate fault domain: Router A → Router C path segment. Supporting evidence: correlated loss/latency across two affected targets, healthy control target on alternate branch, and route observation change. Caveat: traceroute non-response alone does not confirm a failed router; verify with device telemetry or an additional probe.”

This is an illustrative scenario, not measured results. Any numeric thresholds and confidence percentages shown in the final demo must come from documented test data or be labelled as illustrative.

Recommended initial feature set

- Target and probe configuration; recurring scheduled checks.
- Incident grouping and event timeline.
- Topology view with normal / degraded / suspected states.
- Evidence panel that links each claim to timestamped observations.
- Candidate ranking with confidence band and explicit caveats.
- CSV/JSON export for reproducibility and judge review.

Solution Optimization

The optimisation goal is not merely to increase the number of probes. It is to reduce diagnosis time and avoid misleading alerts by combining relevant evidence and making uncertainty visible.

Before vs proposed approach

Dimension	Typical manual workflow	Proposed system	How to validate
Evidence collection	Operator runs separate commands/tools	Scheduled, timestamped probe events in one view	Collection coverage; missing-data rate
Fault localisation	Manual comparison of symptoms and path	Topology-aware candidate ranking	Top-1 / Top-3 localisation accuracy
Explanation	Interpretation may be undocumented	Evidence for/against and caveats shown per candidate	Operator review of explanation correctness
Alert quality	Independent alarms can be noisy	Correlated incidents and configurable thresholds	Precision, recall, false alerts per hour
Auditability	Results scattered across terminals/logs	Stored raw observations and incident timeline	Reproducibility of incident replay
Uncertainty	A failed check may be over-interpreted	Confidence band and “insufficient evidence” state	Calibration; abstention quality

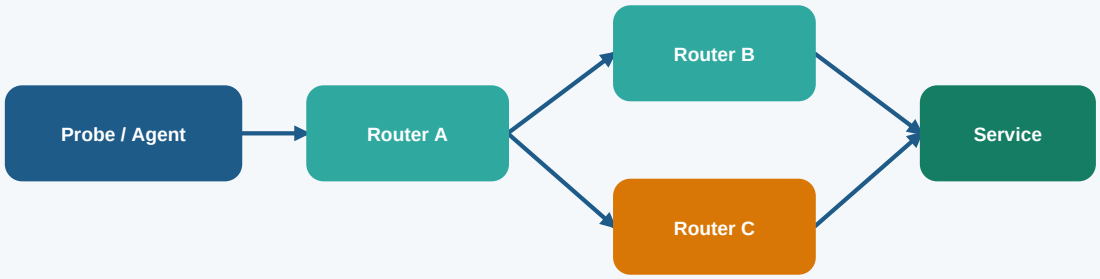
Proposed scoring logic (transparent baseline)

For each candidate link/device c, calculate a weighted evidence score using normalised, recent signals: S(c) = Σ w■ · e■(c) · q■ · r■. Here e■(c) is the support of evidence item i for candidate c, q■ is data quality, r■ is a recency factor, and w■ is a documented configurable weight. Contradictory evidence should subtract support or lower confidence. The score is a ranking aid, not a calibrated probability unless validated against labelled incidents.

Safeguards

- Use baseline-aware thresholds per target/service rather than one universal latency threshold.
- Require persistence or repeated observations before raising high-severity alerts.
- Deduplicate alerts by incident and shared topology context.
- Provide an “unknown / insufficient evidence” outcome when probe coverage is too low.
- Do not claim an exact router/interface without corroborating device-side data or strong multi-point evidence.

Architecture Design (UML / System Architecture)



Illustrative topology: fault is inferred from multiple independent observations, not one failed probe.

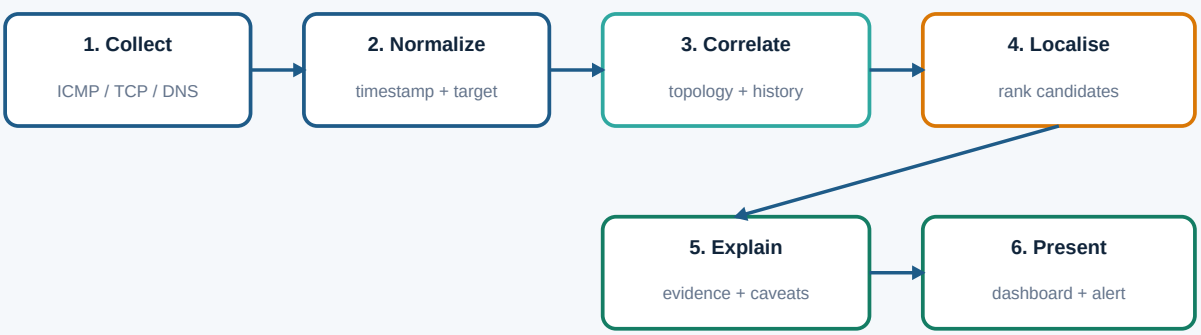
Logical components

Component	Responsibility	Suggested implementation
Probe workers	Run scheduled checks and return raw outcomes	Node.js child processes or dedicated network tools; restrict targets and privileges
Ingestion/API	Validate and accept probe events; manage targets and incidents	Node.js + Express REST API
Correlation engine	Normalise signals, compare baselines, rank candidate faults	Deterministic rules first; ML only after labelled data exists
Topology store	Store devices, links, targets and dependencies	MongoDB documents; graph traversal in application layer for prototype
Dashboard	Show topology, incident timeline, evidence and explanations	React + Tailwind CSS
Observability/security	Logs, health, access control, probe audit	Structured logs, role-based access, allowlist and rate limits

Architecture notes

- Separate probe execution from the web request path; long-running diagnostics should be scheduled/background jobs.
- Store both raw observations and derived incident assessments so results can be replayed and audited.
- Use consistent IDs for targets/devices/links; store timestamps in UTC.
- Keep credentials and tokens in environment-managed secrets; never expose them to the browser.
- For a hackathon demo, a small known topology with simulated faults is safer and easier to reproduce than scanning an uncontrolled network.

Data Flow Diagram (DFD)



Proposed prototype workflow; adapters and device-level telemetry depend on deployment access.

DFD explanation

- **External inputs:** authorised target configuration, topology definition and active-probe responses.
- **Process 1 — Collection:** probe workers execute allowed checks and capture outcomes plus timing metadata.
- **Process 2 — Normalisation:** convert tool-specific output into a stable event schema; failed collection is recorded separately from failed target reachability.
- **Process 3 — Correlation:** align observations by time window and target dependency; compare to baselines and unaffected control paths.
- **Process 4 — Localisation:** score candidate nodes/links, attach evidence references and produce a confidence band or abstain.
- **Data stores:** targets/topology, raw observations, baselines, incident assessments and audit events.
- **Outputs:** dashboard visualisation, alerts, incident export and operator verification steps.

Example event schema

```
eventId, timestampUtc, probeSource, targetId, probeType, status, latencyMs,
packetLossPct, dnsRcode, hopSequence, errorCode, rawOutputRef, qualityFlags
```

Privacy and safety: collect only data needed for troubleshooting, avoid storing payload content, restrict probes to approved assets, apply retention limits, and audit changes to targets and probe schedules.

Key distinction: a probe worker error (permission denied, tool unavailable, timeout in the collector) must not be stored as if the target itself were confirmed down.

Deployment Timeline & Evaluation Plan

A short, milestone-based build plan is suitable for the hackathon. Dates can be adjusted to the event schedule; the stages below describe sequence and exit criteria rather than a guaranteed completion date.

Implementation milestones

Phase	Work items	Exit criterion
1. Scope & design	Define lab topology, target allowlist, event schema, threat/safety boundaries	Reviewed topology and reproducible test cases
2. Probe collection	Implement ICMP/reachability, DNS and TCP/HTTP checks; capture timestamps and errors	Repeatable observations stored through API
3. Backend & storage	Express endpoints, MongoDB models, incident and target APIs	Validated API flow and queryable event history
4. Correlation engine	Baseline thresholds, topology-aware rules, candidate ranking and evidence links	Correct behaviour on scripted fault scenarios
5. Frontend dashboard	Topology status, metrics, alerts, explanation panel, responsive layout	End-to-end incident visible and explainable
6. Test & demo	Run normal/fault cases, record metrics, fix false positives, prepare demo	Evidence-backed demo and documented limitations

Evaluation metrics (report measured values only after testing)

Metric	Definition	Why it matters
Detection precision	TP / (TP + FP)	Share of raised fault alerts that correspond to injected faults
Detection recall	TP / (TP + FN)	Share of injected faults detected
Top-1 / Top-3 localisation	True fault appears at rank 1 / within first 3 candidates	Measures whether ranking narrows the search area
Time to detect (TTD)	Fault injection time to first valid alert	Measures monitoring responsiveness
Time to localise (TTL)	Fault injection time to correct candidate ranking	Measures diagnosis speed
False alerts / hour	Unfounded incident alerts per test hour	Measures alert noise

Test scenarios: endpoint/service down; DNS failure; high latency; packet loss; simulated link outage; traceroute ICMP suppression; collector permission failure; and healthy control branch. Include negative controls so the system is tested against misleading symptoms as well as obvious outages.

Bibliography & Technical References

The following primary references ground the measurement and implementation choices. RFCs are standards-track or protocol specifications; Prometheus Blackbox Exporter is an operational tool, not an IETF standard.

[1] J. Postel, “Internet Control Message Protocol,” RFC 792, Internet Engineering Task Force, September 1981. https://www.rfc-editor.org/info/rfc792/

[2] P. Mockapetris, “Domain Names—Concepts and Facilities,” RFC 1034, Internet Engineering Task Force, November 1987. https://www.rfc-editor.org/info/rfc1034/

[3] P. Mockapetris, “Domain Names—Implementation and Specification,” RFC 1035, Internet Engineering Task Force, November 1987. https://www.rfc-editor.org/info/rfc1035/

[4] S. Shalunov, B. Teitelbaum, A. Karp, J. Boote and M. Zekauskas, “A One-way Active Measurement Protocol (OWAMP),” RFC 4656, September 2006. https://www.rfc-editor.org/info/rfc4656/

[5] K. Hedayat, R. Krzanowski, A. Morton, K. Yum and J. Babiarz, “A Two-Way Active Measurement Protocol (TWAMP),” RFC 5357, October 2008. https://www.rfc-editor.org/info/rfc5357/

[6] Prometheus Authors, “Blackbox Exporter: Probe endpoints over HTTP, HTTPS, DNS, TCP, ICMP and gRPC,” project documentation and configuration. https://github.com/prometheus/blackbox_exporter

[7] Internet Engineering Task Force, “IP Performance Metrics (IPPM) Working Group,” background and measurement standards index. https://datatracker.ietf.org/wg/ippm/about/

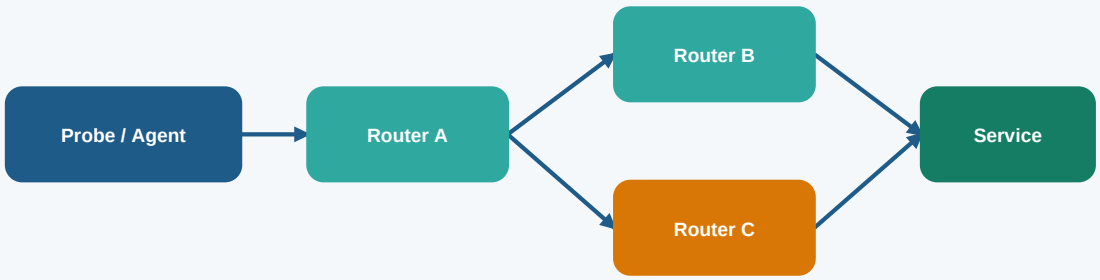
Research and implementation notes

- RFC 792, RFC 1034/1035, RFC 4656 and RFC 5357 define or describe protocol behaviour; they do not validate this project’s scoring model.
- Blackbox Exporter documentation confirms supported probe types and configuration options; whether it fits the final stack depends on the demo environment and permissions.
- The candidate scoring formula, UI, topology model, correlation rules and evaluation plan in this report are proposed project design decisions.
- No performance or accuracy results are claimed here. Insert measured results from the team’s controlled experiments before presenting numerical claims.

Accessed for preparation: 14 October 2026. Verify current RFC status and tool versions before final submission.

Thank You

Evidence first. Hypotheses second. Verification always.



Illustrative topology: fault is inferred from multiple independent observations, not one failed probe.

PS-035 — Evidence-Based Operator Network Fault Localisation

Questions & Discussion

Core promise: explain why a path or component is suspected, show the observations behind that conclusion, and be explicit when the evidence is not enough.